

Disciplína: AI Foundations & Learning Models

Professor: Dheny Rodrigues Fernandes

Grupo 2: Samuel Cupertino, Henrique Arduini, Renato de Jesus Rocha, Ksenia Busquet

Aplicação de modelos de IA/ML no varejo omnicanal - Análise de sentimentos em reviews

1. Introdução e Problemática

A Quantum Commerce consolidou-se como uma gigante nativa digital do varejo omnicanal, operando em 12 países e gerenciando um extenso catálogo superior a 5 milhões de SKUs. Contudo, o rápido crescimento gerou um "muro da escalabilidade operacional", tornando métodos tradicionais obsoletos para manter a qualidade do suporte e a personalização em larga escala.

Diante do desafio proposto pela diretoria executiva para transformar a plataforma em uma operação orientada por inteligência, este projeto aborda o desafio de **Análise de sentimento em reviews**. O objetivo é processar o massivo volume de avaliações de clientes para gerar um "termômetro de satisfação" por produto ou categoria. Esta problemática é crítica, pois com a elevada quantidade de SKUs, é humanamente impossível analisar individualmente cada avaliação para identificar problemas de qualidade ou logística de forma tempestiva.

2. Motivação e Justificativa da Arquitetura

Para resolver o problema de análise de sentimentos em texto de forma precisa, propõe-se a utilização de uma arquitetura de **Deep Neural Networks (Redes Neurais Profundas)** com múltiplas camadas ocultas.

A escolha desta arquitetura justifica-se pela complexidade inerente aos dados de linguagem natural. Modelos matemáticos mais simples, como um único Perceptron, são limitados a resolver problemas lineares. Textos e sentimentos, no entanto, exigem a combinação de múltiplas regiões de decisão para criar fronteiras de separabilidade não-lineares, o que é alcançado através da adição de neurônios e camadas ocultas na rede.

Se utilizássemos abordagens simplistas, incorreríamos no problema de *Underfitting*, onde o modelo seria incapaz de capturar a complexidade semântica das avaliações e geraria resultados muito abaixo do esperado. As Redes Neurais Profundas, por sua vez, permitem o mapeamento de abstrações complexas e são ideais para retornar uma probabilidade associada a cada classe de sentimento (Classificação Multi-classe).

Vale ainda comparar a DNN com duas alternativas mais realistas do que um Perceptron isolado. De um lado, modelos clássicos como Regressão Logística ou Naive Bayes sobre uma

representação TF-IDF são computacionalmente mais baratos e, em corpora de reviews relativamente diretos, costumam atingir acurácia competitiva — por isso são usados neste trabalho como baseline de comparação (Seção 3.4). De outro lado, arquiteturas que preservam a ordem das palavras, como LSTM/GRU, ou modelos pré-treinados baseados em Transformers (ex.: BERT), tendem a superar uma DNN feed-forward sobre bag-of-words justamente por capturarem negação, ironia e dependências de longo alcance no texto — fenômenos comuns em reviews de e-commerce. A DNN foi escolhida como ponto de partida por equilibrar custo computacional, simplicidade de implementação e ganho de desempenho sobre os baselines lineares; a evolução natural da solução em produção seria migrar para um Transformer pré-treinado em português (ex.: BERTimbau) à medida que o volume de dados e a criticidade da aplicação justifiquem o custo adicional.

3. Proposta de Solução

3.1. Arquitetura e Dados

A arquitetura do modelo será composta por uma camada de entrada (*Input Layer*), múltiplas camadas ocultas (*Hidden Layers*) configurando o aprendizado profundo, e uma camada de saída (*Output Layer*). Os dados necessários consistem nas características extraídas dos textos dos *reviews* dos clientes (entrada), enquanto a camada de saída fornecerá os valores de probabilidade associados ao termômetro de satisfação.

Antes da vetorização, os textos passam por um pré-processamento: normalização para caixa baixa, lematização (redução de cada palavra à sua forma-base, via spaCy) e remoção de stopwords em português. Como o scikit-learn não traz uma lista nativa de stopwords para o português, essa lista é obtida da biblioteca spaCy (`spacy.lang.pt.stop_words`), que já disponibiliza um conjunto validado para o idioma sem necessidade de baixar um modelo treinado. O texto limpo é então vetorizado via TF-IDF, gerando a representação numérica consumida pela rede. No notebook demonstrativo, esse pipeline é aplicado sobre uma amostra do B2W-Reviews01, corpus real de avaliações de e-commerce em português coletado do site da Americanas.com, o que aproxima a demonstração do cenário real da Quantum Commerce.

Os rótulos de sentimento não são anotados manualmente: são derivados da nota atribuída pelo próprio cliente no corpus (campo `overall_rating`), mapeando 1 e 2 estrelas para Negativo, 3 para Neutro e 4 e 5 para Positivo. Essa estratégia, conhecida como proxy label, é padrão em bases de reviews por dispensar rotulação especializada em escala, mas carrega uma premissa: a de que a nota reflete fielmente o conteúdo do texto. Na prática existe descolamento, sendo a faixa de 3 estrelas a mais ruidosa, já que reúne tanto avaliações genuinamente neutras quanto textos claramente positivos ou negativos acompanhados de nota intermediária. Essa limitação é retomada na Seção 4.

A distribuição de classes resultante é fortemente desbalanceada, característica conhecida de bases de e-commerce, em que avaliações de 4 e 5 estrelas concentram a maior parte do volume. Sem tratamento, o modelo tenderia a maximizar a acurácia simplesmente prevendo Positivo na maioria dos casos, justamente a classe de menor valor operacional para o caso de uso, já que o interesse do negócio está em detectar insatisfação. Para mitigar isso, aplicamos ponderação de classes no treinamento (`class_weight='balanced'` no baseline e pesos equivalentes na função de erro da DNN) e adotamos o F1-macro como métrica principal de comparação, conforme detalhado na Seção 3.4.

3.2. Pipeline de Treinamento

O treinamento do modelo seguirá um fluxo iterativo e matematicamente otimizado:

- **Feedforward:** Os dados de entrada são multiplicados por matrizes de pesos ($\mathbf{W}$), somados a um limiar ou *Bias* ($\mathbf{b}$), e submetidos a uma função de ativação não-linear (σ). O *Bias* ajuda nas mudanças das funções de ativação e adapta a funcionalidade do neurônio. A predição final ($\hat{y}$) é dada pela aplicação sucessiva destas funções ao longo das camadas.
- **Função de Ativação:** Para as camadas ocultas, utilizaremos a função ReLU em vez da Sigmoide para evitar o problema do *Vanishing Gradient*, situação onde derivadas muito pequenas diminuem o gradiente exponencialmente, impedindo a atualização eficaz dos pesos nas camadas iniciais da rede.
- **Função de Erro (Error Function):** O quão distante a predição está do sentimento real será medido pela função *Log Loss*, comumente utilizada em classificação para minimizar o erro total do modelo.
- **Backpropagation:** O erro será retropropagado utilizando o otimizador **Stochastic Gradient Descent (SGD)**. Calcular o gradiente usando todos os dados (*Batch Gradient Descent*) é ineficiente e pesado para um catálogo de 5 milhões de SKUs. O SGD resolve isso selecionando dados aleatoriamente para atualizar os gradientes a cada época.
- **Fuga de Mínimos Locais:** Para evitar que o gradiente descendente pare em mínimos locais subótimos, aplicaremos as técnicas de *Momentum* (que considera a média dos passos anteriores na descida) e *Random Restart* (iniciando o gradiente em pontos aleatórios).

3.3. Prevenção de Overfitting

Para assegurar que o modelo aprenda a generalizar e não apenas memorize o ruído estatístico do conjunto de treinamento (*Overfitting*), aplicaremos:

- **Regularização L2:** Penalização de pesos elevados na função de erro inserindo o fator

$$\lambda(w_1^2 + \dots + w_n^2).$$

- **Dropout:** Neurônios serão ignorados temporariamente com uma probabilidade p , evitando coadaptações complexas e forçando as camadas a assumirem responsabilidade probabilística de forma distribuída.

- **Early Stopping:** Acompanharemos o *Model Complexity Graph* para interromper o treinamento no ponto ideal (*Just Right*), exatamente quando o erro de teste começar a aumentar em relação ao de treinamento.

3.4. Avaliação e Consumo do Resultado

Para validar objetivamente o modelo antes de disponibilizá-lo em produção, complementamos o acompanhamento do Log Loss durante o treinamento com métricas de classificação no conjunto de teste: Acurácia, F1-macro (mais robusta ao desbalanceamento entre as classes Positivo, Negativo e Neutro) e a matriz de confusão, que evidencia especificamente onde o modelo confunde sentimentos (por exemplo, Neutro sendo classificado como Positivo). Essas métricas são comparadas às obtidas pelo baseline de Regressão Logística citado na Seção 2, tornando explícito, com números, o ganho de desempenho da DNN. O baseline também é validado com Cross-Validation k-fold ($k=5$), técnica vista na disciplina para obter uma estimativa mais robusta de desempenho do que um único split treino/teste; não a aplicamos à DNN pelo custo computacional (ela já treina 5 vezes por conta do Random Restart), usando em seu lugar um conjunto de validação com Early Stopping, prática padrão em Deep Learning.

Em produção, o resultado do modelo alimentaria dois mecanismos concretos: (i) um dashboard de “termômetro de satisfação” segmentado por produto e categoria, atualizado em lote conforme novas avaliações chegam, permitindo aos gestores de categoria monitorar tendências; e (ii) um sistema de alertas automáticos que notifica o time responsável sempre que a proporção de reviews Negativos de um produto recém-lançado ultrapassar um limiar definido dentro de uma janela de tempo curta (ex.: nas primeiras 48 horas após o lançamento), permitindo ação corretiva antes que o problema escale. O modelo seria reprocessado periodicamente (ex.: retreinamento semanal) para incorporar o vocabulário e os padrões de novas avaliações, e disponibilizado às demais equipes via um endpoint de API ou job batch integrado ao pipeline de dados existente da Quantum Commerce.

3.5. Comparação com Embeddings e Transfer Learning (Word2Vec, Attention e BERT)

Para validar empiricamente a comparação feita na Seção 2, o notebook demonstrativo implementa e compara, além do baseline e da DNN, mais três abordagens vistas na disciplina. Primeiro, o Word2Vec (Skip-gram, treinado no próprio corpus de reviews): diferentemente do TF-IDF, que ignora a ordem e o significado das palavras, o Word2Vec aprende embeddings — vetores densos em que palavras com significados semelhantes ficam próximas no espaço vetorial. Cada review é representado pela média dos vetores das suas palavras e classificado com o mesmo baseline de Regressão Logística, isolando o efeito da representação.

Em seguida, o notebook traz uma demonstração conceitual do mecanismo de Attention — $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})V$ —, base dos Transformers e do BERT: diferentemente de embeddings estáticos como o Word2Vec, cada palavra passa a “olhar” para as demais

palavras da frase e ponderar sua relevância para o contexto, o que é essencial para capturar negação e ironia em reviews (ex.: “não gostei” x “gostei”).

Por fim, aplicamos Transfer Learning com o BERTimbau (BERT pré-treinado em português) nas duas estratégias vistas na Aula de Fine-Tuning: (i) Feature Extraction, com o backbone do BERT totalmente congelado — extraímos o embedding do token [CLS] de cada review e treinamos apenas um classificador leve sobre ele, sem risco de catastrophic forgetting; e (ii) Fine-Tuning parcial, descongelando as últimas camadas do encoder com learning rate baixa ($2e-5$), célula opcional por ser mais custosa computacionalmente. Os resultados de todas as seis abordagens (baseline, DNN, Word2Vec, BERT feature extraction, BERT fine-tuning e a comparação entre elas) são consolidados num comparativo final no notebook (Seção 11), incluindo gráfico de F1-macro por abordagem.

4. Considerações e Potencial de Impacto

Limitações e Riscos: Um dos riscos na construção da rede é a definição incorreta da Taxa de Aprendizado (*Learning Rate*); passos muito grandes impedem a convergência do gradiente, enquanto passos muito curtos atrasam drasticamente o treinamento. É necessário monitorar rigorosamente o processo para garantir o equilíbrio entre *Underfitting* e *Overfitting* através dos dados de teste.

Além dos riscos ligados ao treinamento, três limitações de escopo devem ser explicitadas:

Qualidade do rótulo. Conforme descrito na Seção 3.1, os rótulos derivam da nota do cliente e não de anotação especializada. O teto de desempenho do modelo é, portanto, limitado pela consistência entre nota e texto, e parte do erro observado na matriz de confusão reflete ruído do próprio rótulo, não incapacidade da arquitetura.

Cobertura de idiomas. A demonstração foi conduzida integralmente em português, tanto no corpus quanto no BERTimbau, enquanto a operação da Quantum Commerce abrange 12 países. A extensão aos demais mercados exigiria corpora rotulados por idioma ou a adoção de um encoder multilíngue, como o XLM-RoBERTa, evitando o custo de manter e monitorar um modelo por idioma.

Granularidade da análise. A classificação é feita no nível do review, produzindo uma polaridade única por avaliação. Um texto como "produto excelente, mas a entrega atrasou três semanas" é reduzido a um único rótulo, ainda que contenha julgamentos opostos sobre produto e logística. A evolução natural da solução é a Análise de Sentimento Baseada em Aspecto (Aspect-Based Sentiment Analysis), que separa esses eixos e permite direcionar o alerta descrito na Seção 3.4 ao time correto, qualidade ou operação logística, em vez de sinalizar apenas insatisfação genérica.

Potencial de Valor: A arquitetura proposta entregará o exigido "termômetro de satisfação" de forma escalável e automática. Os resultados do modelo apoiarão decisões críticas de negócio, permitindo aos gerentes da Quantum Commerce identificar anomalias na qualidade de produtos recém lançados instantaneamente, impulsionar itens altamente avaliados e sustentar uma experiência do cliente de excelência em meio a um catálogo de milhões de itens.